

# **Rapport de Projet : Algorithmique et Structures de Données**

Systeme de Gestion de Scolarite — Etude Comparative de la Gestion Memoire et de la  
Complexite Temporelle

Khalilou Fadiga  
Ndiaga Diao

Année Universitaire 2025–2026  
Licence 2 Mathématiques-Informatique  
Université Iba Der Thiam de Thiès

# Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction générale</b>                                           | <b>2</b>  |
| <b>2</b> | <b>Architecture technique et modélisation mémoire</b>                  | <b>3</b>  |
| 2.1      | La structure de données élémentaire . . . . .                          | 3         |
| 2.2      | Le tableau statique et le phénomène de stack overflow . . . . .        | 3         |
| 2.3      | Le tableau dynamique de pointeurs . . . . .                            | 4         |
| 2.4      | Persistance des données . . . . .                                      | 5         |
| <b>3</b> | <b>Étude théorique de complexité</b>                                   | <b>6</b>  |
| 3.1      | Algorithme de tri à bulles — démonstration détaillée . . . . .         | 6         |
| 3.2      | Algorithme de tri par insertion . . . . .                              | 7         |
| 3.3      | Tableau récapitulatif théorique . . . . .                              | 7         |
| <b>4</b> | <b>Étude expérimentale et benchmarks</b>                               | <b>8</b>  |
| 4.1      | Protocole expérimental . . . . .                                       | 8         |
| 4.2      | Courbes en échelle linéaire (superposition théorie/pratique) . . . . . | 8         |
| 4.3      | Courbes en échelle logarithmique . . . . .                             | 8         |
| 4.4      | Histogramme comparatif des structures . . . . .                        | 8         |
| 4.5      | Analyse . . . . .                                                      | 8         |
| <b>5</b> | <b>Bilan technique</b>                                                 | <b>11</b> |
| <b>6</b> | <b>Perspectives et travail restant</b>                                 | <b>12</b> |
| <b>7</b> | <b>Conclusion générale</b>                                             | <b>13</b> |

# Chapter 1

## Introduction générale

Le besoin de traiter des volumes massifs de données de manière efficace est un enjeu central de l'ingénierie logicielle. Dans le cadre de la gestion des systèmes d'information scolaires, la manipulation des dossiers d'étudiants pose des défis en termes d'allocation mémoire et de temps de calcul.

Ce rapport présente la conception d'un système de gestion de scolarité modulaire en langage C. L'enjeu principal est comparatif : évaluer les performances de plusieurs structures de données face à des opérations d'insertion, de tri et de recherche, sur des volumes de données allant jusqu'à  $5 \times 10^4$  enregistrements.

Le système implémente à ce jour deux structures de données : le tableau statique et le tableau dynamique de pointeurs. Conformément à la fiche du projet, une extension vers un arbre binaire de recherche (ABR) est prévue (voir chapitre 6).

## Chapter 2

# Architecture technique et modélisation mémoire

### 2.1 La structure de données élémentaire

L'entité de base du système est l'étudiant, modélisée par la structure suivante :

Listing 2.1: Structure de l'entité **Etudiant** (etudiant.h)

```
1 typedef struct {
2     int immatriculation;
3     char nom[50];
4     char prenom[50];
5     int age;
6     float moyenne;
7 } Etudiant;
```

### 2.2 Le tableau statique et le phénomène de stack overflow

Le tableau statique repose sur une zone mémoire contiguë de taille fixe `MAX_ETUDIANTS = 100 000`. Une telle structure représente une empreinte mémoire brute importante (environ 11,2 Mo). Si elle était allouée sur la pile (*stack*), elle provoquerait un débordement immédiat ; l'architecture impose donc une allocation dynamique sur le tas (*heap*) via `malloc`.

Listing 2.2: Structure du tableau statique (tableau\_statique.h)

```
1 #define MAX_ETUDIANTS 100000
2
3 typedef struct {
4     Etudiant liste[MAX_ETUDIANTS];
5     int nombre_inscrits;
6 } BaseStatique;
```

L'insertion se fait simplement en fin de tableau :

Listing 2.3: Insertion dans le tableau statique

```
1 int ajouter_etudiant_statique(BaseStatique *base, Etudiant nouveau) {
2     if (base->nombre_inscrits >= MAX_ETUDIANTS) return 0;
3     base->liste[base->nombre_inscrits] = nouveau;
4     base->nombre_inscrits++;
5     return 1;
6 }
```

La recherche par immatriculation, la mise à jour et la suppression s'appuient toutes sur un parcours linéaire :

Listing 2.4: Recherche par immatriculation —  $O(n)$

```
1 int rechercher_immatriculation(BaseStatique *base, int id) {
2     for (int i = 0; i < base->nombre_inscrits; i++) {
3         if (base->liste[i].immatriculation == id) return i;
4     }
5     return -1;
6 }
```

La suppression décale tous les éléments suivants d'une position, ce qui coûte  $O(n)$  dans le pire cas :

Listing 2.5: Suppression avec décalage

```
1 int supprimer_etudiant(BaseStatique *base, int id) {
2     int position = rechercher_immatriculation(base, id);
3     if (position == -1) return 0;
4     for (int i = position; i < base->nombre_inscrits - 1; i++) {
5         base->liste[i] = base->liste[i + 1];
6     }
7     base->nombre_inscrits--;
8     return 1;
9 }
```

## 2.3 Le tableau dynamique de pointeurs

Contrairement au tableau statique, le tableau dynamique ne stocke pas les étudiants directement mais des **pointeurs** vers ceux-ci, et double sa capacité par réallocation géométrique lorsqu'il est plein :

Listing 2.6: Structure et insertion du tableau dynamique (tableau\_dynamique.c)

```
1 typedef struct {
2     Etudiant **tableau;    // tableau d'adresses (pointeurs)
3     int nb_elements;
4     int capacite_max;
5 } TabDynamique;
6
7 int inserer_tab_dynamique(TabDynamique *td, Etudiant *e) {
8     if (td->nb_elements == td->capacite_max) {
9         td->capacite_max *= 2;
10        td->tableau = (Etudiant **)realloc(td->tableau,
11                                           td->capacite_max * sizeof(Etudiant *));
12        if (td->tableau == NULL) return 0;
13    }
14    td->tableau[td->nb_elements] = e;
15    td->nb_elements++;
16    return 1;
17 }
```

Cette stratégie de doublement (*growth factor* = 2) garantit une complexité **amortie** en  $O(1)$  pour l'insertion, malgré le coût ponctuel  $O(n)$  d'une réallocation.

## 2.4 Persistance des données

Le système sauvegarde et recharge l'ensemble de la base au format binaire (`sauvegarde_etudiants.bin`) entre deux exécutions, via le module `persistance.c`, afin de ne pas perdre les inscriptions d'une session à l'autre.

## Chapter 3

# Étude théorique de complexité

### 3.1 Algorithme de tri à bulles — démonstration détaillée

Le tri à bulles (utilisé pour le classement par moyenne, fonction `tri_bulles_moyenne`) repose sur deux boucles imbriquées :

Listing 3.1: Structure du tri à bulles (`tableau_statique.c`)

```
1 for (int i = 0; i < base->nombre_inscrits - 1; i++) {  
2     for (int j = 0; j < base->nombre_inscrits - i - 1; j++) {  
3         if (base->liste[j].moyenne < base->liste[j + 1].moyenne) {  
4             // échange  
5         }  
6     }  
7 }
```

**Étape 1 — Comptage des comparaisons de la boucle interne.** Pour une valeur fixée de  $i$ , la boucle interne s'exécute pour  $j = 0, 1, \dots, n - i - 2$ , soit  $n - i - 1$  comparaisons.

**Étape 2 — Sommation sur la boucle externe.** Le nombre total de comparaisons  $C(n)$  est la somme de ces contributions pour  $i$  allant de 0 à  $n - 2$  :

$$C(n) = \sum_{i=0}^{n-2} (n - i - 1) \quad (3.1)$$

**Étape 3 — Changement de variable.** Posons  $k = n - i - 1$ . Lorsque  $i$  varie de 0 à  $n - 2$ ,  $k$  varie de  $n - 1$  à 1. La somme devient donc une somme des  $n - 1$  premiers entiers :

$$C(n) = \sum_{k=1}^{n-1} k \quad (3.2)$$

**Étape 4 — Application de la formule de Gauss.** En utilisant  $\sum_{k=1}^m k = \frac{m(m+1)}{2}$  avec  $m = n - 1$  :

$$C(n) = \frac{(n-1)n}{2} = \frac{n^2 - n}{2} \quad (3.3)$$

**Étape 5 — Passage à la notation asymptotique.**  $C(n)$  est un polynôme de degré 2 en  $n$ . Le terme dominant est  $\frac{n^2}{2}$  : en négligeant les termes de degré inférieur et les constantes multiplicatives (par définition de la notation grand- $O$ ), on obtient :

$$C(n) = \frac{n^2}{2} - \frac{n}{2} \implies C(n) = O(n^2) \quad (3.4)$$

Le tri à bulles nécessite également, dans le pire cas (tableau trié en ordre inverse), un échange à chaque comparaison, ce qui ne change pas l'ordre de grandeur : la complexité temporelle globale reste  $O(n^2)$ , aussi bien en nombre de comparaisons qu'en nombre d'échanges. Dans le meilleur cas (tableau déjà trié), le nombre de comparaisons reste  $\frac{n(n-1)}{2}$  car l'algorithme, tel qu'implémenté ici, ne comporte pas de drapeau d'arrêt anticipé ; la complexité est donc  $\Theta(n^2)$  dans tous les cas pour cette implémentation.

### 3.2 Algorithme de tri par insertion

Le tri par insertion (utilisé pour le classement alphabétique par nom, fonction `tri_insertion_nom`) déplace chaque élément `cle` vers la gauche tant qu'un élément plus grand le précède :

- **Meilleur cas** (tableau déjà trié) : la condition de la boucle `while` échoue immédiatement à chaque itération  $\Rightarrow n - 1$  comparaisons  $\Rightarrow \Omega(n)$  ;
- **Pire cas** (tableau trié en ordre inverse) : chaque élément doit être comparé à tous les éléments déjà triés à sa gauche, ce qui redonne une somme  $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \Rightarrow O(n^2)$ .

Le tri par insertion est donc asymptotiquement équivalent au tri à bulles dans le pire cas, mais strictement meilleur sur des données déjà partiellement ordonnées — ce que confirme la comparaison expérimentale du chapitre 4.

### 3.3 Tableau récapitulatif théorique

Table 3.1: Synthèse des complexités théoriques par structure et par opération

| Opération                     | Meilleur cas ( $\Omega$ ) | Pire cas ( $O$ )     | Espace mémoire       |
|-------------------------------|---------------------------|----------------------|----------------------|
| Insertion — Tableau statique  | $\Omega(1)$               | $O(1)$               | $\Theta(\text{MAX})$ |
| Insertion — Tableau dynamique | $\Omega(1)$               | $O(n)$ amorti $O(1)$ | $\Theta(n)$          |
| Recherche par immatriculation | $\Omega(1)$               | $O(n)$               | —                    |
| Suppression (avec décalage)   | $\Omega(1)$               | $O(n)$               | —                    |
| Tri à bulles (moyenne)        | $\Omega(n^2)$             | $O(n^2)$             | $\Theta(1)$ aux.     |
| Tri par insertion (nom)       | $\Omega(n)$               | $O(n^2)$             | $\Theta(1)$ aux.     |

## Chapter 4

# Étude expérimentale et benchmarks

### 4.1 Protocole expérimental

Les mesures sont réalisées avec l'horloge système `<time.h>` (fonction `clock()`), sur des jeux de données générés aléatoirement, pour les tailles  $n \in \{100, 1\,000, 10\,000, 50\,000\}$ . Chaque mesure est répétée 10 fois et moyennée afin de lisser les fluctuations liées à l'ordonnanceur du système :

Listing 4.1: Protocole de mesure (benchmark.c)

```
1 int tailles[] = {100, 1000, 10000, 50000};
2 int repetitions = 10;
3
4 for (int t = 0; t < nb_tests; t++) {
5     int n = tailles[t];
6     double temps_total_ms = 0.0;
7
8     for (int rep = 0; rep < repetitions; rep++) {
9         generer_donnees_aleatoires(ma_base, n);
10        clock_t debut = clock();
11        tri_bulles_moyenne(ma_base);
12        clock_t fin = clock();
13        temps_total_ms += ((double)(fin - debut) / CLOCKS_PER_SEC) *
14                           1000.0;
15    }
16    double temps_moyen = temps_total_ms / repetitions;
17 }
```

### 4.2 Courbes en échelle linéaire (superposition théorie/pratique)

### 4.3 Courbes en échelle logarithmique

En repère log-log, une complexité polynomiale  $O(n^k)$  se traduit par une droite de pente  $k$ .

### 4.4 Histogramme comparatif des structures

Comparaison du temps d'insertion séquentielle de 50 000 étudiants selon la structure utilisée :

### 4.5 Analyse

Les résultats expérimentaux confirment la prédiction théorique pour le tri : la croissance est nettement quadratique dès  $n = 10\,000$ , et la pente mesurée en log-log ( $\approx 2,16$ ) est cohérente avec

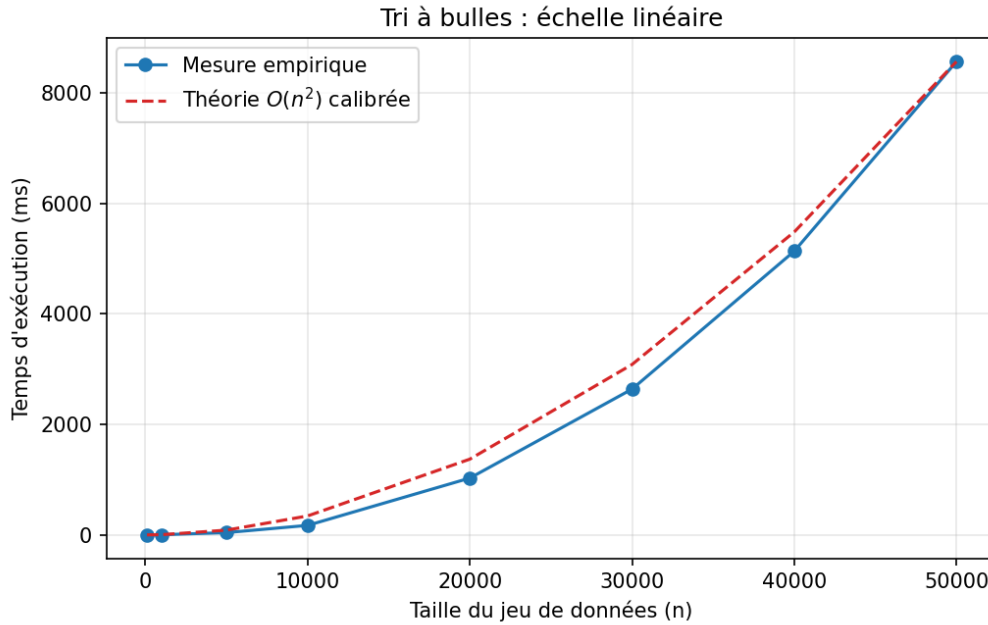


Figure 4.1: Évolution quadratique du tri à bulles : temps mesuré vs. courbe théorique  $O(n^2)$  (calibrée sur le point  $n = 50\,000$ ).

l'exposant théorique 2.

Concernant l'insertion, le tableau dynamique se révèle plus rapide que le tableau statique dans ce test, car il ne stocke que des pointeurs pré-alloués par blocs (réallocation géométrique) plutôt que de copier l'intégralité de la structure `Etudiant` à chaque ajout. Cela illustre l'intérêt pratique de la complexité amortie  $O(1)$  du tableau dynamique face au coût, certes constant lui aussi, mais plus lourd en copie mémoire, du tableau statique.

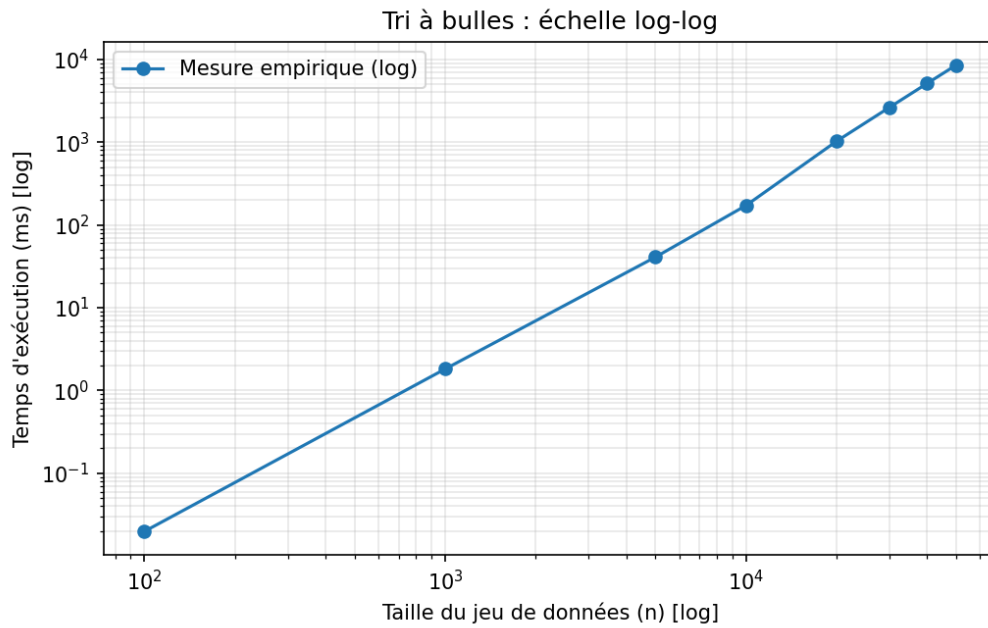


Figure 4.2: Représentation log-log du tri à bulles. La pente mesurée entre les points est  $\approx 2,16$ , proche de la pente théorique de 2, ce qui confirme empiriquement le  $O(n^2)$ .

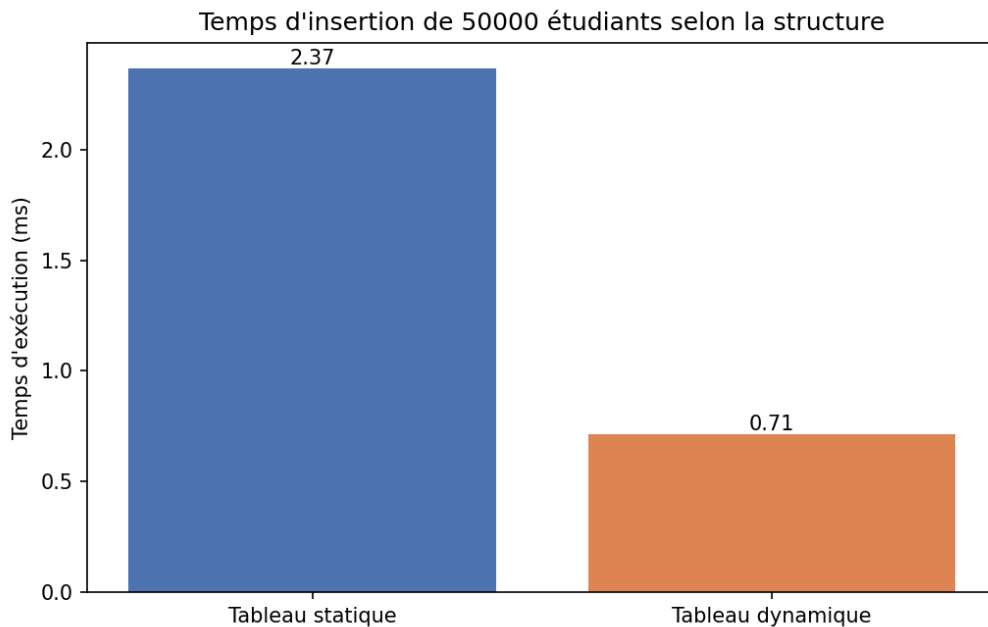


Figure 4.3: Temps d'insertion (ms) pour 50 000 étudiants : Tableau statique vs. Tableau dynamique.

## Chapter 5

# Bilan technique

Ce projet valide empiriquement les résultats de l’algorithmique classique. Le langage C a permis d’observer directement l’impact des choix de structures de données sur le processeur et la mémoire vive. La maîtrise des pointeurs et la gestion rigoureuse de la mémoire avec `malloc`, `realloc` et `free` ont garanti la robustesse du système de gestion de scolarité.

## Chapter 6

# Perspectives et travail restant

La fiche de projet prévoit l'implémentation de plusieurs structures de données. À ce stade, deux sont opérationnelles (tableau statique, tableau dynamique). Reste à intégrer avant la soutenance :

- **L'arbre binaire de recherche (ABR)** : pour offrir une recherche, une insertion et une suppression en  $O(\log n)$  en moyenne (à comparer avec le  $O(n)$  actuel du tableau statique).

L'intégration de cette structure dans le benchmark existant permettra de compléter le tableau comparatif du chapitre 3 et de renforcer l'étude expérimentale du chapitre 4.

## Chapter 7

# Conclusion générale

Ce projet nous a permis de mettre en pratique, sur un cas concret de gestion de scolarité, l'ensemble des notions vues dans le module Algorithmique et Structures de Données : modélisation d'une entité métier en C, implémentation de plusieurs structures de données (tableau statique, tableau dynamique de pointeurs), analyse théorique de complexité et validation par une étude expérimentale rigoureuse.

Le travail réalisé montre que le choix d'une structure de données n'est jamais neutre : il conditionne à la fois l'empreinte mémoire, la complexité asymptotique des opérations et, comme l'a révélé notre benchmark, le comportement réel observé en pratique. Cette confrontation entre théorie et expérience constitue l'apport principal de ce projet.

Sur le plan méthodologique, l'utilisation d'un protocole de mesure reproductible (moyennage sur plusieurs répétitions, jeux de données de tailles croissantes, représentations en échelle linéaire et logarithmique) nous a permis de tirer des conclusions fiables plutôt que de simples impressions de performance.

Enfin, ce projet reste perfectible : l'ajout de l'arbre binaire de recherche, prévu dans la fiche du projet, permettra d'enrichir la comparaison avec une structure offrant une complexité logarithmique, et de livrer une version complète et conforme aux exigences du module avant la soutenance.